PRACTICAL - 02

Part A – Practice Programs (Hour 1)

PROBLEM1	**1. Smart thermostat (Thermostat.java, main in this file)** (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)". (e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)". (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount(). **Expected output:** *temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.*

CODE	public class Thermostat {
	  private String location;
	  private int temperature;
	
	  private static final int MIN = 16;
	  private static final int MAX = 30;
	  private static int activeCount = 0;
	
	  public Thermostat(String location, int startTemp) {
	  this.location = location;
	  if (startTemp >= MIN && startTemp <= MAX)
	  temperature = startTemp;
	  else
	  temperature = 22;
	  activeCount++;
	  }
	
	  public Thermostat(String location) {
	  this(location, 22);

```
    }

    public void raise() {
        if (temperature < MAX)
            temperature++;
        else
            System.out.println("Already at maximum (30)");
    }

    public void lower() {
        if (temperature > MIN)
            temperature--;
        else
            System.out.println("Already at minimum (16)");
    }

    public int getTemperature() {
        return temperature;
    }

    public static int getActiveCount() {
        return activeCount;
    }

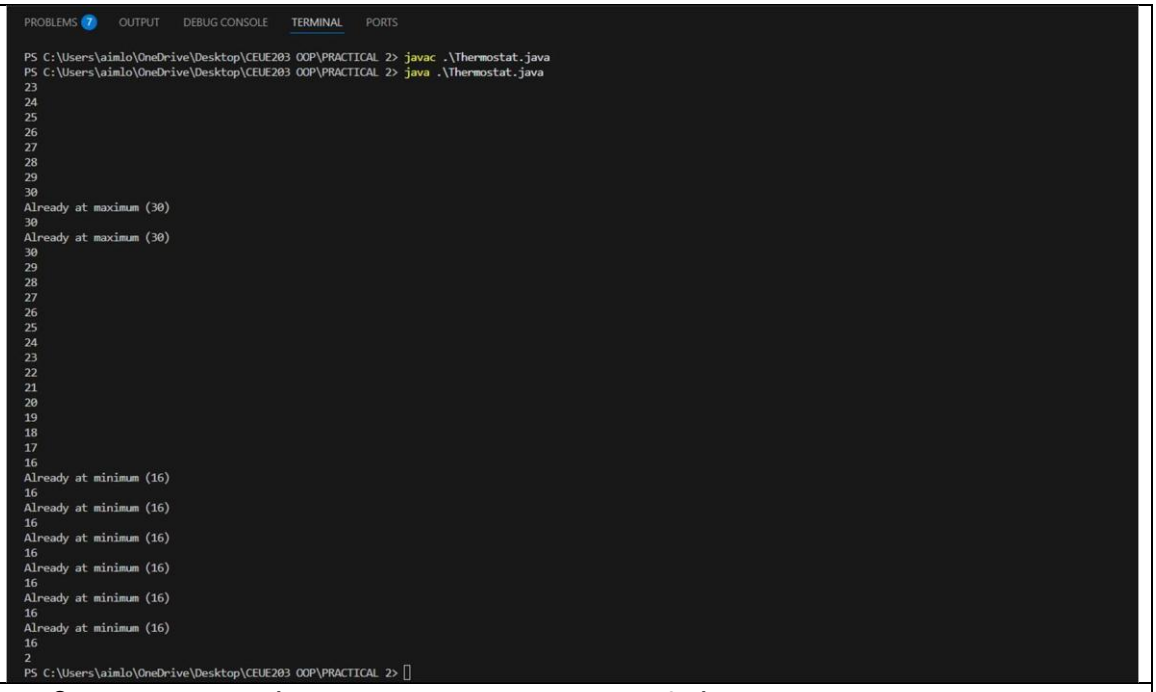
    public static void main(String[] args) {
        Thermostat t1 = new Thermostat("Living Room", 22);
        Thermostat t2 = new Thermostat("Bedroom");

        for (int i = 0; i < 10; i++) {
            t1.raise();
            System.out.println(t1.getTemperature());
        }

        for (int i = 0; i < 20; i++) {
            t1.lower();
            System.out.println(t1.getTemperature());
        }

        System.out.println(Thermostat.getActiveCount());
    }
}
```

OUTPUT



PROBLEM2

2. Cinema show (CinemaShow.java, main in this file)

(a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked = 0.

(b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100).

(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.

(d) cancel(int n): increase seatsAvailable by n but never above capacity.

(e) Add getSeatsAvailable() and static getTotalBooked().

(f) In main, run a sequence of book() and cancel() calls, printing the result and seats after each, then print getTotalBooked().

Expected Output: Booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

CODE

```
public class CinemaShow {
   private String title;
   private int seatsAvailable;
   private final int capacity;
   private static int totalBooked = 0;

   public CinemaShow(String title, int capacity) {
      this.title = title;
      this.capacity = capacity;
      seatsAvailable = capacity;
   }

   public CinemaShow(String title) {
      this(title, 100);
   }

    public boolean book(int n) {
      if (n <= seatsAvailable) {
         seatsAvailable -= n;
          totalBooked += n;
         return true;
      }
      return false;
   }

   public void cancel(int n) {
      seatsAvailable += n;
      if (seatsAvailable > capacity)
         seatsAvailable = capacity;
   }

   public int getSeatsAvailable() {
      return seatsAvailable;
   }

   public static int getTotalBooked() {
      return totalBooked;
   }

   public static void main(String[] args) {
      CinemaShow show = new CinemaShow("Avengers", 50);

      System.out.println(show.book(20));
      System.out.println(show.getSeatsAvailable());

      System.out.println(show.book(15));
      System.out.println(show.getSeatsAvailable());
```

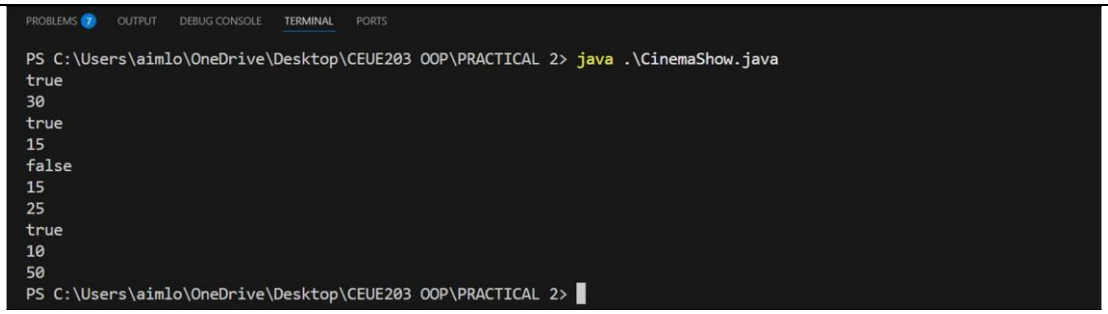
```
        System.out.println(show.book(20));
        System.out.println(show.getSeatsAvailable());

        show.cancel(10);
        System.out.println(show.getSeatsAvailable());

        System.out.println(show.book(15));
        System.out.println(show.getSeatsAvailable());

        System.out.println(CinemaShow.getTotalBooked());
    }
}
```

OUTPUT



PROBLEM3

3. (For Advanced Learners – home) Parking lot (ParkingLot.java)

(a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue = 0.

(b) park(String type): for "two", if twoWheelers < twoCap increment and add 20 to revenue; for "four", if fourWheelers < fourCap increment and add 40 to revenue; otherwise print "Full" and reject.

(c) leave(String type): decrement the matching count but never below 0.

(d) In main, simulate a sequence of park() and leave() events, printing each outcome, then print final occupancy and revenue.

Expected Output: Parking into a full section is refused; revenue = 20 × (two parked) + 40 × (four parked).

CODE

```
public class ParkingLot {
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
```

```
    this.twoCap = twoCap;
    this.fourCap = fourCap;
  }

  public void park(String type) {
    if (type.equals("two")) {
      if (twoWheelers < twoCap) {
        twoWheelers++;
        revenue += 20;
        System.out.println("Two-wheeler  parked");
      } else {
        System.out.println("Full");
      }
    } else if (type.equals("four")) {
      if (fourWheelers < fourCap) {
        fourWheelers++;
        revenue += 40;
        System.out.println("Four-wheeler  parked");
      } else {
        System.out.println("Full");
      }
    }
  }

  public void leave(String type) {
    if (type.equals("two")) {
      if (twoWheelers > 0)
        twoWheelers--;
    } else if (type.equals("four")) {
      if (fourWheelers > 0)
        fourWheelers--;
    }
  }

  public static void main(String[] args) {
    ParkingLot p = new ParkingLot(2, 2);

    p.park("two");
    p.park("two");
    p.park("two");

    p.park("four");
    p.park("four");
    p.park("four");

    p.leave("two");
    System.out.println("Two Wheelers: " + p.twoWheelers);
```

```
        p.leave("four");
        System.out.println("Four Wheelers: " + p.fourWheelers);

        System.out.println("Revenue: " + revenue);
    }
}
```

OUTPUT

```
PROBLEMS 7  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 2> javac   Open file in editor (ctrl + click)
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 2> java .\ParkingLot.java
Two-wheeler parked
Two-wheeler parked
Full
Four-wheeler parked
Four-wheeler parked
Full
Two Wheelers: 1
Four Wheelers: 1
Revenue: 120
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 2>
```

Part B — Project Work (Hour 2): Mini Bank

PROBLEM

MiniBank is the console banking application you will build across the whole semester. In this first lab you set up your tools and create the project and its menu shell — the screen the user navigates. Every later lab attaches one real banking feature behind these menu options.

1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-; add the faculty as a collaborator.
2. Create a public class named MiniBank that contains the main method.
3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header.
4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer.
5. Display the numbered menu to the user and read the chosen number with a Scanner.
6. Use a switch expression on the choice to print a short placeholder for that option (for example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message.

CODE

```
public class MiniBank {

    public static void main(String[] args) {

        Customer c1 = new Customer("Om Sutariya", "om@gmail.com", "9930030060");

        Account[] accounts = new Account[3];
```

```
        accounts[0] = new Account(c1.getName(), 5000);
        accounts[1] = new Account("Bhavy", 3000);
        accounts[2] = new Account("Manthan");

        accounts[0].deposit(1000);
        accounts[0].withdraw(2000);

        accounts[1].deposit(500);
        accounts[1].withdraw(1000);

        accounts[2].deposit(2000);
        accounts[2].withdraw(500);

        for (Account a : accounts) {
          System.out.println(a.getAccountNumber() + " " + a.getOwnerName() + " Balance: " +
a.getBalance());
        }
    }
}

class Customer {

    private  String  name;
    private  String  email;
    private String mobile;
    private final String customerId;

    private static long customerCounter = 101;

    private static String generateCustomerId() {
        return "CUST" + customerCounter++;
    }

    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.customerId = generateCustomerId();
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
```

```
    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }
}

class Account {

    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber() {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long balance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = balance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public void deposit(long amount) {
        if (amount < 0)
            return;
        balance += amount;
    }

    public boolean withdraw(long amount) {
        if (amount < 0)
            return false;

        if (balance >= amount) {
            balance -= amount;
            return true;
```

```
        }
        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }
}
```

OUTPUT

